LOGIC INTELLIGENCE TECHNOLOGIES

Where Logic Meets Innovation

OFFICIAL STANDARD OPERATING PROCEDURE

The 50-Point Enterprise Deployment Checklist

Standard Operating Procedure for Production-Grade AI Architectures

DOCUMENT CONTROL

PREPARED & AUTHORISED UNDER Vikash Saravanan — Founder & Lead Engineer	ORGANISATION Logic Intelligence Technologies Pvt. Ltd.
DOCUMENT TYPE Technical Deployment Checklist · Engineering SOP	VERSION 1.0
SCOPE 8 phases · 50 checkpoints	STATUS Approved · In force

APPROVED & AUTHORISED FOR USE

This Standard Operating Procedure is approved, controlled and authorised for use across all Logic Intelligence Technologies engineering engagements.

Vikash Saravanan — Founder & Lead Engineer

CERTIFIED & AUTHORISED BY

AUTHORISED SIGNATURE

DOCUMENT CONTENTS · EIGHT PHASES

1	Architecture & Scope Validation	1–5	2	Frontend Engineering & UX	6–12
3	Backend & Database Integrity	13–19	4	AI & Data Pipeline Integration	20–27
5	Telephony & Real-Time WebSockets	28–32	6	Security & Compliance	33–38
7	QA, Testing & Telemetry	39–44	8	DevOps, CI/CD & Handoff	45–50

DOCUMENT PURPOSE & APPLICABILITY

Purpose

This document is the engineering release-review standard used by Logic Intelligence Technologies (LIT) when designing, building, testing, securing and deploying production web applications and AI-powered platforms. It defines fifty specific, testable controls across eight software-development lifecycle phases, each with an explicit acceptance condition and the evidence a technical reviewer should inspect.

APPLICABILITY

It applies to LIT web applications and AI-powered platforms across their delivery lifecycle. Checkpoints that fall outside a project's scope — for example telephony on a system with no voice features — are marked Not Applicable with a documented rationale rather than removed. Where a checkpoint names a specific host or provider (Vercel, Render, Supabase, a model API), the equivalent control applies when a different approved provider is selected. Numerical targets are recommended engineering targets or project-defined acceptance thresholds unless a checkpoint states they are verified measurements.

APPROVAL & AUTHORISATION

This Standard Operating Procedure is the official, controlled engineering document of Logic Intelligence Technologies. It is approved, in force and authorised for use across the organisation's engineering engagements. Changes are version-controlled and re-authorised by the approving authority before release.

DOCUMENT STATUS

Approved · In force · Version 1.0. Certified and authorised by Vikash Saravanan, Founder & Lead Engineer, Logic Intelligence Technologies Pvt. Ltd. This controlled document is maintained under version control and supersedes any earlier draft.

EIGHT-PHASE TECHNICAL OVERVIEW

How the checklist is organised

PHASE	LIFECYCLE PHASE	POINTS	COUNT
1	Architecture & Scope Validation	1–5	5
2	Frontend Engineering & UX	6–12	7
3	Backend & Database Integrity	13–19	7
4	AI & Data Pipeline Integration	20–27	8
5	Telephony & Real-Time WebSockets (If Applicable)	28–32	5
6	Security & Compliance	33–38	6
7	QA, Testing & Telemetry	39–44	6
8	DevOps, CI/CD & Handoff	45–50	6
	Total	1–50	50

HOW TO USE THIS CHECKLIST

Each checkpoint has an engineering requirement, a measurable acceptance condition and the evidence a reviewer inspects. Mark each box only when the evidence exists. Checkpoints outside a project's scope are marked Not Applicable with a rationale; unresolved critical issues block release at the sign-off stage.

1

PHASE 1

Architecture & Scope Validation

POINTS 1–5

5 checkpoints

Technical discovery, repository architecture, cloud provisioning, infrastructure ownership and environment configuration.

☐ 01. Architecture & Requirements Baseline

ENGINEERING REQUIREMENT

Produce a documented system architecture covering frontend applications, backend services, databases, authentication, external APIs, network boundaries and deployment targets, with functional and non-functional requirements (performance, availability, data sensitivity) traced to it.

ACCEPTANCE CRITERIA

An approved architecture document exists with current diagrams, a requirements traceability matrix, named non-functional targets and an identified dependency list; scope is signed off by the technical lead.

VERIFICATION EVIDENCE

Architecture document and diagrams, the traceability matrix, and the dated scope approval in the project record.

☐ 02. GitHub Repository & Branch Protection

ENGINEERING REQUIREMENT

Define the repository strategy (monorepo or multirepo), set CODEOWNERS, protect the main and release branches, and require passing status checks and pull-request review before merge; adopt a release-tagging convention.

ACCEPTANCE CRITERIA

main cannot be pushed to directly; merges require at least one approving review and green required checks; releases are tagged; repository access follows least privilege.

VERIFICATION EVIDENCE

Branch-protection settings, CODEOWNERS file, a sample merged pull request showing enforced reviews/checks, and the repository access list.

☐ 03. Environment Separation (Dev / Staging / Production)

ENGINEERING REQUIREMENT

Provision isolated development, staging and production environments with environment-scoped secrets, independent deployment targets and distinct data stores so non-production code cannot write to production records.

ACCEPTANCE CRITERIA

Each environment has its own configuration and credentials; a staging action cannot create or modify a production row; promotion between environments is explicit and controlled.

VERIFICATION EVIDENCE

Environment configuration matrix, secret-scope settings per environment, and a test showing a staging write does not reach production data.

☐ 04. Cloud Infrastructure & Account Provisioning

ENGINEERING REQUIREMENT

Verify ownership and configuration of Vercel projects, Render services, Supabase projects, DNS, required service accounts and billing limits, with an explicit mapping between environments, repositories, services and domains.

ACCEPTANCE CRITERIA

Every environment maps to a known project, service, repository and domain under LIT-controlled accounts with billing limits set; no orphaned or personal-account resources serve production.

VERIFICATION EVIDENCE

The environment-to-service mapping table, provider project/service identifiers, DNS ownership records, and configured billing/spend limits.

☐ 05. Technical Risk & Dependency Assessment

ENGINEERING REQUIREMENT

Maintain a register of external-service dependencies, operational risks, expected traffic, data sensitivity, availability requirements, expected costs and mitigations, including failure scenarios and recovery ownership.

ACCEPTANCE CRITERIA

Each external dependency has a documented failure mode, mitigation and named owner; availability and cost expectations are recorded and agreed.

VERIFICATION EVIDENCE

The risk and dependency register with owners, mitigations and recovery responsibilities.

2 PHASE 2 — Frontend Engineering & UX

POINTS 6–12 · 7 checkpoints

Production-quality frontend development, consistent design, accessibility, performance and trustworthy interactions.

☐ 06. Next.js & React Production Architecture

ENGINEERING REQUIREMENT

Validate the rendering strategy (server vs client boundaries), route structure, error boundaries, caching and metadata configuration, and confirm a clean production build. Where Next.js 16 is adopted, use its supported application architecture and build requirements.

ACCEPTANCE CRITERIA

A production build completes without errors or disallowed warnings; server/client boundaries are deliberate; error boundaries and metadata are present on key routes.

VERIFICATION EVIDENCE

The production build log, the route/rendering map, and a review of error-boundary and metadata configuration.

☐ 07. TypeScript & Component Architecture

ENGINEERING REQUIREMENT

Enforce strict TypeScript, reusable typed components, explicit interface contracts, predictable state management and validation of external data at boundaries; avoid unsafe any and unvalidated inputs.

ACCEPTANCE CRITERIA

The type-check passes in strict mode with no new errors; shared components have typed props; external responses are validated before use.

VERIFICATION EVIDENCE

The strict type-check (tsc) output in CI, and a code review of component interfaces and boundary validation.

☐ 08. Tailwind Design Tokens & Brand Consistency

ENGINEERING REQUIREMENT

Define a central design system — typography, spacing, colour, components, responsive breakpoints, visual states and accessible contrast — as reusable tokens rather than values duplicated across pages.

ACCEPTANCE CRITERIA

Colour, type and spacing come from shared tokens; components reuse them; text/background pairs meet contrast targets; no ad-hoc hard-coded style values on key screens.

VERIFICATION EVIDENCE

The design-token/config source, and a review showing components consuming tokens plus contrast-check results.

☐ 09. Responsive & Cross-Browser Compatibility

ENGINEERING REQUIREMENT

Test navigation, forms, tables, dialogs, animations and interactive elements at representative desktop, tablet and mobile viewports across the supported browser set.

ACCEPTANCE CRITERIA

Key journeys render and function correctly at each defined breakpoint on each supported browser, with no broken layout, overflow or inoperable control.

VERIFICATION EVIDENCE

A device/browser test matrix with pass/fail per journey and screenshots of the key breakpoints.

☐ 10. Core Web Vitals & Frontend Performance

ENGINEERING REQUIREMENT

Measure Largest Contentful Paint, Interaction to Next Paint and Cumulative Layout Shift against documented thresholds, and verify image optimization, JavaScript bundle budgets, caching and route-loading behaviour.

ACCEPTANCE CRITERIA

Key pages meet the project's stated LCP, INP and CLS thresholds in real-browser measurement; bundle sizes are within budget; images are optimized.

VERIFICATION EVIDENCE

Lab and/or field Web Vitals reports for key routes, and the bundle-analysis output against the defined budget.

☐ 11. Accessibility & Inclusive Interaction

ENGINEERING REQUIREMENT

Apply WCAG 2.2 AA-oriented checks: keyboard navigation, screen-reader names, form-error association, focus management, contrast, reduced-motion support and responsive text scaling, combining automated and manual testing.

ACCEPTANCE CRITERIA

Important journeys are fully operable by keyboard and screen reader; automated checks report no critical violations; focus order and error messaging are correct.

VERIFICATION EVIDENCE

Automated accessibility scan results plus a manual audit record for the primary journeys.

☐ 12. Forms, API Wiring & User Feedback

ENGINEERING REQUIREMENT

Every production form validates inputs, submits to an authorized backend, persists where required, guards against duplicate submission and shows truthful success or failure states reflecting the real result.

ACCEPTANCE CRITERIA

Submitting a form creates the expected backend record and routes to its operational destination; failures are shown as failures; a success message is never displayed without persistence.

VERIFICATION EVIDENCE

A staging submission trace from form to stored record/notification, and a test confirming failure states on a forced backend error.

3

PHASE 3

Backend & Database Integrity

POINTS 13–19

7 checkpoints

Secure APIs, PostgreSQL architecture, data validation, tenant isolation and transaction reliability.

13. FastAPI Service Architecture & API Contracts

ENGINEERING REQUIREMENT

Verify FastAPI routing, request/response schemas, status codes, authentication dependencies, exception handling and generated OpenAPI documentation, with contracts agreed between frontend and backend.

ACCEPTANCE CRITERIA

Endpoints validate against declared schemas, return correct status codes, enforce auth dependencies and return structured errors; the OpenAPI spec matches the implemented contract.

VERIFICATION EVIDENCE

The OpenAPI document, schema/validation tests, and a review of auth dependencies and error handlers.

14. PostgreSQL Schema Design & Referential Integrity

ENGINEERING REQUIREMENT

Model data with correctly typed identifiers, primary and foreign keys, constraints and indexes, normalized where appropriate, and manage changes through versioned migrations.

ACCEPTANCE CRITERIA

The deployed schema matches migrations; foreign-key and uniqueness constraints are enforced; lookup columns are indexed; no orphaned references exist in test data.

VERIFICATION EVIDENCE

Migration history, a schema diff against the database, and constraint/integrity test results.

15. Supabase Row-Level Security & Tenant Isolation

ENGINEERING REQUIREMENT

Define explicit Row-Level Security policies for every table exposed through Supabase APIs, enforcing least privilege and tenant separation, and keep the service-role key server-side only.

ACCEPTANCE CRITERIA

A user from one tenant cannot read or write another tenant's rows via the API; privileged functions are protected; the service-role key never reaches the browser.

VERIFICATION EVIDENCE

The RLS policy definitions, a cross-tenant access test (expected denials), and confirmation the service-role key is absent from client bundles.

☐ 16. Connection Pooling & Resource Management

ENGINEERING REQUIREMENT

Configure connection pooling, connection lifecycle, transaction timeouts, retry behaviour and connection limits appropriate to the serverless/container hosting model.

ACCEPTANCE CRITERIA

Under expected concurrency the service stays within connection limits without exhaustion; timeouts and retries behave as configured.

VERIFICATION EVIDENCE

Pooler configuration, and a concurrency test report showing connection counts and error rates within limits.

☐ 17. Idempotent Operations & Duplicate Prevention

ENGINEERING REQUIREMENT

Apply idempotency keys, unique constraints and transactional processing so retries of critical submissions, payment events, provisioning actions and background jobs do not create duplicates.

ACCEPTANCE CRITERIA

Replaying the same request or event produces exactly one effect; concurrent duplicates are rejected or de-duplicated.

VERIFICATION EVIDENCE

An idempotency test (duplicate/retried requests), and the unique-constraint definitions on the affected tables.

☐ 18. Transaction Integrity, Backup & Recovery

ENGINEERING REQUIREMENT

Verify ACID transaction boundaries, scheduled backups, a tested restoration procedure and migration-rollback planning; a successful backup is not treated as proof of recoverability.

ACCEPTANCE CRITERIA

A backup can be restored into a usable environment and verified; transaction boundaries prevent partial writes; a migration can be rolled back.

VERIFICATION EVIDENCE

A dated restore-test record showing data verified after restore, plus the backup schedule and rollback plan.

☐ 19. Server-Side Validation & Auditability

ENGINEERING REQUIREMENT

Validate all API inputs on the server, generate authoritative identifiers server-side, and emit durable audit events for privileged operations; never rely on frontend-only persistence or success.

ACCEPTANCE CRITERIA

Invalid inputs are rejected server-side; identifiers are server-generated; privileged actions produce audit records with actor, action and timestamp.

VERIFICATION EVIDENCE

Validation test cases, and sample audit-log entries for privileged operations.

4 PHASE 4 — AI & Data Pipeline Integration

POINTS 20–27

8 checkpoints

Production-grade AI integrations, reliable retrieval, security, latency, data governance and output quality.

☐ 20. AI Provider Selection & Credential Management

ENGINEERING REQUIREMENT

Document the chosen model provider(s) — for example an approved Meta model API service or Google Vertex AI — and verify credentials, model availability, quotas, regional requirements and failure handling.

ACCEPTANCE CRITERIA

The provider and model are recorded with region and quota; credentials are stored securely; the system degrades gracefully when the provider errors or is rate-limited.

VERIFICATION EVIDENCE

The provider/model configuration record, secret-store references, and a test of provider-failure handling.

☐ 21. Knowledge Ingestion & Data Provenance

ENGINEERING REQUIREMENT

Ingest only authorized documents, with validation, source metadata, content checksums, duplicate detection, update handling and deletion procedures; do not fabricate or pad the corpus.

ACCEPTANCE CRITERIA

Every ingested item has a verifiable source and checksum; duplicates are detected; updates and deletions propagate to the index.

VERIFICATION EVIDENCE

The ingestion log with sources and checksums, and a deletion/update test showing the index reflects the change.

☐ 22. pgvector Semantic Search & Index Validation

ENGINEERING REQUIREMENT

Where semantic retrieval is used, validate embedding generation, vector dimensions, the pgvector schema, the chosen vector index and both search accuracy and query performance.

ACCEPTANCE CRITERIA

Embedding dimensions match the index; retrieval returns relevant results on a labelled set above the project's relevance threshold; query latency is within target.

VERIFICATION EVIDENCE

A retrieval benchmark with relevance scores and latency, and the pgvector schema/index definition.

☐ 23. Retrieval-Augmented Generation Architecture

ENGINEERING REQUIREMENT

Implement document chunking, retrieval ranking, permission filtering, source attribution, context assembly and citation integrity so the assistant cannot surface content a user is not authorized to see.

ACCEPTANCE CRITERIA

Answers cite retrieved sources accurately; permission filters exclude unauthorized documents from context; citations resolve to real passages.

VERIFICATION EVIDENCE

A permission-filtering test (authorized vs unauthorized user on the same query), and a citation-integrity review.

☐ 24. Prompt Injection & Tool-Use Protection

ENGINEERING REQUIREMENT

Separate trusted instructions from retrieved content, test for prompt injection, enforce tool permissions and require confirmation gates for consequential actions; untrusted documents must not override authorization.

ACCEPTANCE CRITERIA

Injection attempts embedded in documents do not change system behaviour or trigger unauthorized tool use; consequential actions require explicit confirmation.

VERIFICATION EVIDENCE

A prompt-injection test suite with results, and the tool-permission and confirmation-gate configuration.

☐ 25. AI Response Quality & Factual Verification

ENGINEERING REQUIREMENT

Maintain evaluation datasets covering factual accuracy, completeness, consistency, citation quality and hallucination resistance; time-sensitive facts must come from appropriately fresh, verifiable sources.

ACCEPTANCE CRITERIA

The model meets the project's stated quality thresholds on the evaluation set; unsupported claims and stale facts are caught by the evaluation.

VERIFICATION EVIDENCE

The evaluation dataset and scored results, including hallucination and citation-quality metrics.

☐ 26. AI Latency, Streaming & Resource Optimization

ENGINEERING REQUIREMENT

Measure time to first token, completion time, retrieval latency, model execution time and error rates; apply streaming, bounded context windows, caching and resource controls, and define per-application SLOs.

ACCEPTANCE CRITERIA

Measured latencies meet the application-specific SLOs; streaming is used where appropriate; context size and cost stay within bounds.

VERIFICATION EVIDENCE

A latency/throughput report against the defined SLOs, and the streaming/context/caching configuration.

☐ 27. AI Privacy, Memory & Retention Controls

ENGINEERING REQUIREMENT

Control storage of conversation history with consent where appropriate, per-user isolation, retention policies and deletion; where zero-retention is required, verify both application behaviour and the provider's contractual and technical retention settings.

ACCEPTANCE CRITERIA

Conversation data is isolated per user and retained/deleted per policy; a deletion request removes the data; any zero-retention claim is backed by the provider's configured settings.

VERIFICATION EVIDENCE

The retention policy, a user-data deletion test, and the provider's documented retention/opt-out configuration.

5

PHASE 5

POINTS 28–32

Telephony & Real-Time WebSockets (If Applicable)

5 checkpoints

Voice assistants, streaming audio, telephony APIs and low-latency conversation pipelines. For projects with no voice/telephony scope, retain these five and mark them Not Applicable with a documented rationale.

☐ 28. Audio Capture, Sampling & Encoding Validation

ENGINEERING REQUIREMENT

For speech pipelines, verify microphone capture, sampling-rate conversion (for example 16 kHz mono PCM where the selected STT provider requires it), frame sizes, encoding and handling of invalid audio; do not force 16 kHz when the provider contract specifies another format.

ACCEPTANCE CRITERIA

Audio reaches the STT provider in its required format; malformed audio is handled without crashing; the sampling rate matches the provider contract.

VERIFICATION EVIDENCE

The audio-pipeline configuration, and a test log showing accepted formats and graceful handling of invalid input.

☐ 29. Non-Blocking Real-Time Audio Pipeline

ENGINEERING REQUIREMENT

Ensure non-blocking I/O, bounded asyncio.Queue (or equivalent) handling, backpressure management, defined queue-overflow behaviour and no blocking work inside latency-sensitive event loops.

ACCEPTANCE CRITERIA

Under sustained audio the event loop is not blocked; queues stay bounded with defined overflow behaviour; no dropped-frame cascade occurs.

VERIFICATION EVIDENCE

A load test of the audio path showing queue depth and loop latency, and a review confirming no blocking calls in hot paths.

☐ 30. WebSocket Session Lifecycle & Reliability

ENGINEERING REQUIREMENT

Require authenticated session establishment, connection-health/heartbeat handling, reconnect behaviour, message sequencing, cleanup and correct behaviour when connectivity is interrupted.

ACCEPTANCE CRITERIA

Sessions authenticate before use; dropped connections reconnect and resume or fail cleanly; no leaked sessions remain after disconnect.

VERIFICATION EVIDENCE

A connection-interruption test showing reconnect/cleanup, and the heartbeat/sequencing configuration.

31. STT LLM TTS Orchestration

ENGINEERING REQUIREMENT

Validate streaming transcription, AI processing, speech generation, interruptibility, cancellation, conversation state and recovery from provider errors, measuring each stage independently.

ACCEPTANCE CRITERIA

A caller can interrupt and the pipeline cancels in-flight work; each stage recovers from a provider error; conversation state stays consistent.

VERIFICATION EVIDENCE

Per-stage latency and error measurements, and an interruption/cancellation test record.

32. End-to-End Voice Latency & Quality

ENGINEERING REQUIREMENT

Define and test response latency, transcription accuracy, speech quality, interruptions and jitter under realistic conditions; a sub-400 ms turnaround may be investigated for suitable streaming components where feasible, but must not be claimed end-to-end unless defined, measured and achieved under documented conditions.

ACCEPTANCE CRITERIA

Measured latency and quality meet the project's documented targets under realistic load; any sub-400 ms claim is scoped to the measured component and conditions.

VERIFICATION EVIDENCE

The end-to-end voice test report with latency distribution, accuracy and the conditions under which it was measured.

6 PHASE 6 — Security & Compliance

POINTS 33–38 · 6 checkpoints

Secure engineering, identity controls, protected data, dependency risk and compliance evidence.

33. Authentication, Authorization & MFA

ENGINEERING REQUIREMENT

Verify secure login, account lifecycle, role-based permissions, MFA where appropriate, session expiry, access revocation and enforcement of authorization at backend boundaries (not only in the UI).

ACCEPTANCE CRITERIA

Access decisions are enforced server-side per role; sessions expire and can be revoked; MFA is available where required; revoked access takes effect immediately.

VERIFICATION EVIDENCE

Authorization test cases per role, session-expiry/revocation tests, and the MFA configuration.

☐ 34. API Rate Limiting & Abuse Protection

ENGINEERING REQUIREMENT

Implement and test rate limits, request-size limits, bot mitigation, brute-force protection and safe throttling of expensive AI endpoints.

ACCEPTANCE CRITERIA

Abusive request volumes are throttled with correct status codes; oversized requests are rejected; expensive endpoints have stricter limits.

VERIFICATION EVIDENCE

Rate-limit configuration and a test showing throttling behaviour and response codes under burst load.

☐ 35. Secrets Management & Credential Rotation

ENGINEERING REQUIREMENT

Hold credentials only in approved secret stores or deployment environments, enable repository secret scanning, minimize access, define rotation procedures and prevent secret leakage through build artifacts and logs.

ACCEPTANCE CRITERIA

No secrets exist in the repository or build output; secret scanning is active; a documented rotation procedure exists and access is least-privilege.

VERIFICATION EVIDENCE

Secret-scanning results, the secret-store configuration, and the rotation procedure.

☐ 36. TLS, DNS, Security Headers & Transport Protection

ENGINEERING REQUIREMENT

Verify HTTPS with valid certificates, appropriate TLS configuration, secure cookies, HSTS where appropriate, a content security policy and essential browser security headers.

ACCEPTANCE CRITERIA

All traffic is HTTPS with valid certificates; security headers are present and correct; cookies are secure and scoped.

VERIFICATION EVIDENCE

An external TLS/headers scan of the production domain, and the header/cookie configuration.

☐ 37. Sensitive Data Protection & Encryption

ENGINEERING REQUIREMENT

Require encryption in transit, encryption at rest where needed, access-controlled object storage, data minimization and retention enforcement, and document which components process sensitive data.

ACCEPTANCE CRITERIA

Sensitive data is encrypted in transit and at rest where required; storage access is controlled; retention is enforced; the data-flow for sensitive payloads is documented.

VERIFICATION EVIDENCE

The data-flow/classification document, storage access-control settings, and encryption configuration evidence.

☐ 38. Vulnerability & Compliance Review

ENGINEERING REQUIREMENT

Run dependency vulnerability scanning and application security testing, cover authentication/authorization test cases, perform a privacy-impact review where appropriate, assign incident-response responsibilities and document remediation; reference regulatory frameworks only when genuinely applicable.

ACCEPTANCE CRITERIA

Known critical/high vulnerabilities are remediated or formally accepted; security test cases pass; incident responsibilities are assigned; no unfounded compliance claims are made.

VERIFICATION EVIDENCE

The dependency-scan and security-test reports with remediation status, and the incident-response ownership record.

7 PHASE 7 — QA, Testing & Telemetry

POINTS 39–44

6 checkpoints

Functional verification, engineering quality, observability and production reliability.

☐ 39. Automated Unit & Integration Testing

ENGINEERING REQUIREMENT

Maintain unit tests, tests with mocked external dependencies, backend integration tests, database tests and frontend component tests, with project-appropriate coverage and critical-path acceptance thresholds.

ACCEPTANCE CRITERIA

The test suite passes in CI; coverage meets the agreed threshold; critical paths have explicit tests.

VERIFICATION EVIDENCE

The CI test run and coverage report against the defined threshold.

☐ 40. End-to-End User Journey Verification

ENGINEERING REQUIREMENT

Test authentication, forms, account activation, dashboard access, API interactions, notifications and any payment or AI workflows against staging, asserting real outcomes rather than only HTTP 200.

ACCEPTANCE CRITERIA

Each critical journey completes end to end on staging with the expected data and side effects, not merely a successful response code.

VERIFICATION EVIDENCE

End-to-end test results per journey, including the resulting records/notifications verified.

41. Error Boundaries & Failure Recovery

ENGINEERING REQUIREMENT

Verify frontend error boundaries, backend exception handling, retry limits, fallback states, circuit breakers and recovery when a provider is unavailable; never show simulated success when a critical dependency fails.

ACCEPTANCE CRITERIA

When a dependency fails, the system shows an honest failure or degraded state and recovers when the dependency returns; retries are bounded.

VERIFICATION EVIDENCE

A fault-injection test record showing behaviour with a forced dependency outage and recovery.

42. Observability, Logging & Error Monitoring

ENGINEERING REQUIREMENT

Implement structured logs with correlation IDs, actionable error monitoring (for example Sentry where selected), tracing where supported and incident alerts, while keeping secrets and unnecessary personal data out of logs.

ACCEPTANCE CRITERIA

Errors surface in monitoring with enough context to act; logs are structured and correlated; no secrets or unnecessary PII appear in logs.

VERIFICATION EVIDENCE

Sample structured log entries with correlation IDs, the monitoring/alert configuration, and a log-redaction check.

43. Performance, Load & Scalability Testing

ENGINEERING REQUIREMENT

Run representative performance, load, stress and concurrency tests, defining throughput, latency, resource utilization and acceptable error thresholds for realistic usage.

ACCEPTANCE CRITERIA

The system meets its throughput/latency targets at expected load and degrades predictably beyond it, within the defined error threshold.

VERIFICATION EVIDENCE

The load-test report showing throughput, latency percentiles, resource use and error rates against targets.

44. Security Regression & Production-Readiness Testing

ENGINEERING REQUIREMENT

Maintain regression tests for access control, database policies, API abuse prevention and sensitive-data exposure, and record known limitations; block release for unresolved critical issues.

ACCEPTANCE CRITERIA

Security regression tests pass; previously fixed issues do not recur; no unresolved critical issue remains open at release.

VERIFICATION EVIDENCE

The security regression results and the open-issues list with severity and release-blocker status.

8 PHASE 8 — DevOps, CI/CD & Handoff

POINTS 45–50

6 checkpoints

Repeatable deployments, automation, infrastructure verification, monitoring and client handover.

45. GitHub Actions CI/CD Pipeline Validation

ENGINEERING REQUIREMENT

Require linting, type checking, automated tests, security checks and build verification in CI, gating deployment so changes reach the production branch only after the pipeline passes.

ACCEPTANCE CRITERIA

A failing check blocks merge/deploy; the production branch only advances through a green pipeline; the workflow is protected from unauthorized edits.

VERIFICATION EVIDENCE

The CI workflow definition and a run history showing gating on lint, types, tests, security and build.

46. Vercel Frontend Production Deployment

ENGINEERING REQUIREMENT

Verify project ownership, Git mapping, build settings, environment variables, the deployed commit, the production domain and post-deployment smoke tests; where Vercel is not the host, document the equivalent checks for the approved provider.

ACCEPTANCE CRITERIA

The production frontend serves the intended commit on the correct domain with correct environment configuration; smoke tests pass after deploy.

VERIFICATION EVIDENCE

The deployment record (project, commit, domain), the environment-variable inventory, and smoke-test results.

47. Render Backend Production Deployment

ENGINEERING REQUIREMENT

Verify service configuration, start commands, environment configuration, database connectivity, any background workers, health checks, resource allocation and the running revision; where Render is not selected, perform the equivalent backend-hosting checks.

ACCEPTANCE CRITERIA

The backend runs the intended revision, connects to the database, passes health checks and has required workers running within allocated resources.

VERIFICATION EVIDENCE

The service configuration, health-check status, running-revision identifier and a database-connectivity check.

48. DNS, Domains, CDN & Search Visibility

ENGINEERING REQUIREMENT

Verify DNS records, domain ownership, TLS certificates, redirects, canonical URLs, sitemap accessibility, robots directives and CDN behaviour, and submit to search engines where applicable, without promising indexing time or ranking.

ACCEPTANCE CRITERIA

The production domain resolves over valid TLS with correct redirects and canonicals; the sitemap and robots rules are correct; CDN caching behaves as intended.

VERIFICATION EVIDENCE

DNS/TLS verification output, the canonical/redirect map, and the sitemap/robots configuration.

49. Rollback & Operational Monitoring

ENGINEERING REQUIREMENT

Maintain documented release identifiers, a tested rollback procedure, monitoring dashboards, alert routing and post-release observation, including migration-recovery considerations; test representative recovery rather than only documenting it.

ACCEPTANCE CRITERIA

A release can be rolled back to a known-good revision in a test; dashboards and alerts are live; post-release monitoring is defined.

VERIFICATION EVIDENCE

A rollback-test record, the monitoring dashboard and alert-routing configuration, and release-identifier history.

☐ 50. IP, Documentation & Client Handover

ENGINEERING REQUIREMENT

Produce a handover inventory covering source repositories, technical documentation, API specifications, deployment architecture, approved access transfer, support responsibilities and intellectual-property terms, without disclosing passwords or private keys; define ownership and transfer per the actual contract.

ACCEPTANCE CRITERIA

The client receives a complete, accurate handover package; access is transferred through approved channels; IP and support terms match the signed contract; no secrets appear in the handover document.

VERIFICATION EVIDENCE

The signed-off handover inventory, the access-transfer record, and the contract clause defining IP ownership and support.

DEPLOYMENT-READINESS SIGN-OFF

Release Review & Decision

This section is completed per project at release review and recorded against this approved Standard Operating Procedure. It is the formal release-decision record for the engagement.

REVIEW SUMMARY

FIELD	ENTRY
Project name	
Review date	
Technical reviewer	
Total applicable checks	
Completed checks	
Not-applicable checks	
Blocked checks	
Open critical issues	

RELEASE DECISION

DECISION	CONDITION
GO	All applicable mandatory release blockers resolved and evidenced
CONDITIONAL GO	Only documented, accepted non-critical exceptions remain
NO-GO	One or more unresolved critical issues or missing mandatory evidence

RELEASE DECISION

☐ GO ☐ CONDITIONAL GO ☐ NO-GO

Reviewer signature / approval

Date

MANDATORY BLOCKERS

A GO decision requires every applicable mandatory release blocker — authentication/authorization, tenant isolation (RLS), server-side validation, backup-restore verification, security regression and CI/CD gating — to be resolved and evidenced.